

gps-lora firmware

Source: firmware/README.md, not yet committed, generated 2026-10-07

Two Arduino firmwares for the ESP32-S3 board in `hardware/`. Build one board with each:

Firmware	Folder	What it does
Tracker (transmitter)	<code>tracker/</code>	Logs a CSV record per GNSS epoch (1 Hz) to microSD, shows status on the OLED, sends position over LoRa every 10 s
Receiver	<code>receiver/</code>	Listens for trackers. Logs every packet to microSD, prints it as JSON on USB, shows distance and direction to the tracker on the OLED. Never transmits on LoRa

Both share one library, `lib/gps-lora-core/`, so GNSS, SD, display, WiFi service mode (web page, log download, OTA update) and the console behave the same on both.

Everything a tracker sends or logs is encrypted and authenticated (AES-128-CCM, one key per tracker). Only holders of your master key can read positions. The full design, threat model, packet and log formats and test vectors are in [docs/SECURITY.md](#) (PDF). Set up keys before first use: [Provisioning](#).

Status: compiles, not yet run on hardware. The board has not been fabricated. Expect fixes during bring-up.

Build and flash

Needs [arduino-cli](#) 1.1 or later. Core, library versions, board options and the shared library are pinned in each folder's `sketch.yaml`, so no setup beyond `arduino-cli` is needed:

```
cd firmware/tracker          # or firmware/receiver
arduino-cli compile --export-binaries .
arduino-cli upload -p /dev/ttyACM0 .
arduino-cli monitor -p /dev/ttyACM0
```

`--export-binaries` puts `tracker.ino.bin` (or `receiver.ino.bin`) under `build/` for OTA uploads; `build/` is git-ignored. Edits to shared code go in `lib/gps-lora-core/src/gpslora/` and apply to both firmwares.

The board enumerates as a USB Serial/JTAG device. If firmware has wedged USB, hold BOOT, tap RESET, release BOOT to force download mode.

Pinned	Version
esp32:esp32 core	3.3.8 (ESP-IDF 5.5)
TinyGPSPlus	1.0.3
SdFat	2.3.0
RadioLib	7.8.1
U8g2	2.36.19

Layout

Path	Contents
<code>tracker/tracker.ino</code>	Tracker: track logging, LoRa transmit, WiFi/LoRa interlock

Path	Contents
receiver/receiver.ino	Receiver: LoRa receive, RX log, JSON output
lib/gps-lora-core/src/gps_lora.h	Umbrella header the sketches include

Shared modules, in `lib/gps-lora-core/src/gpslora/`:

File	Contents
app.*	Start-up and loop shared by both: GNSS epochs, button, VBUS sense, LED, display, watchdog, OTA confirm
role.*	Which firmware this is (set by the sketch)
board.h	Pin map, checked against the schematic netlist
config.h	Tunables: GNSS baud, LoRa frequency/power/SF, intervals, WiFi timeouts
gnss.*	SAM-M10Q: UBX-CFG-VALSET setup, NMEA parsing, PPS, link recovery
storage.*	microSD on SPI2, <code>L0Gnnnn.CSV</code> / <code>RXnnnn.CSV</code> , sync per record, thread safe
lora_link.*	RFM95W via RadioLib on SPI3: TX with duty-cycle limit, or RX
packet.*	Encrypted LoRa packet v2: seal/open, plain C++, host-tested
crypto.*	AES-128-CCM, HMAC/HKDF-SHA256, boot self-test; <code>crypto_mbedtls.cpp</code> is the device backend (hardware AES)
keys.*	Key hierarchy: master → device → radio/log keys
seclog.*	Encrypted SD log lines
security.*	Key storage, message counter, receiver replay protection
net.*	WiFi service mode, web server and OTA, in its own task
timekeeping.*	UTC system clock from GNSS or NTP, SD file timestamps
settings.*	NVS: WiFi credentials, web password
console.*	USB serial commands
snapshot.*	Thread-safe copies of GNSS and RX state for display and web
display.*	SSD1306 status screen, tracker and receiver layouts

Host tests: `make -C firmware/tests` (needs `libssl-dev` and `pip install cryptography`). They run the crypto known-answer tests, check that every single-byte change to a packet is rejected, and require the firmware's packets and log lines to match an independent Python implementation byte for byte. `tools/gps_lora.py` is the PC side: keys, log decryption, packet decoding.

Hardware rules the firmware enforces

From `docs/PCB_DESIGN.md`, "Firmware consequence":

- **SD sync after every record.** No battery, so every power-off is unclean.
- **LoRa at 866.0 MHz**, compile-time check for 865-867 MHz. 14 dBm default; airtime capped at 10%, or 1% if power is set to 20 dBm (RFM95W rating).
- **LoRa RESET (GPIO48) only low or high-Z.** RadioLib's own reset drives it high, so RadioLib gets no reset pin and the module is reset with an open-drain pulse instead.
- **GNSS reset via UBX-CFG-RST.** GPIO7 RESET_N is pulsed (open drain) only after repeated reconfiguration fails, because it forces a cold start.
- **GPIO15 (TIMEPULSE) is an input with no pull.**

- **OLED:** RES# pulse, then U8g2's SSD1306 init, which uses the panel's internal DC/DC values (8Dh 14h, 81h CFh, D9h F1h, AFh).
- **GPIO38-40 are never configured.**
- **WiFi and LoRa never transmit together** (supply budget). The WiFi driver transmits whenever it likes, so per-packet coordination is impossible. Instead the modes exclude each other: while WiFi is on, the tracker sends no LoRa packets, and WiFi only starts once a packet already on air has finished.

GNSS setup

At each boot the receiver is configured in the RAM layer only, so a power cycle always returns it to factory defaults (9600 baud):

- UART1 to 115200 baud
- NMEA GGA and RMC on; GLL, GSA, GSV, VTG off

The config is sent at both 9600 and 115200, because after an ESP32-only reset the receiver is still at 115200. If no valid NMEA arrives for 3 s, it is reconfigured. After 3 failed attempts, RESET_N is pulsed.

A record is written when GGA arrives. u-blox sends it after RMC in each epoch, so every field comes from the same epoch. Fixes with HDOP above 5 or older than 1.5 s are not logged.

Tracker log format

With a key installed, LOGnnnn.CSV is encrypted, one authenticated record per line (format: [SECURITY.md §9](#)). Decrypt on a PC:

```
tools/gps_lora.py decrypt-log ~/gps-lora.master LOG0001.CSV > track.csv
```

```
utc_date,utc_time,lat,lon,alt_m,speed_kmh,course_deg,sats,hdop
2026-10-07,09:12:44,12.9715987,77.5945627,920.4,0.35,0.0,14,0.8
```

Damaged (power cut), altered, duplicated or reordered lines are reported and skipped. Without a key the tracker writes that plaintext CSV directly and warns on the console and OLED.

LoRa packet, version 2

Raw LoRa (not LoRaWAN): 866.0 MHz, BW 125 kHz, SF9, CR 4/5, sync word 0x12, CRC on. 34 bytes, about 247 ms on air (2.5% duty at the 10 s interval):

```
[ver 1][node 2][counter 4] [time, lat, lon, alt, speed, sats, hdop, flags: 19] [tag 8]
clear, authenticated      AES-128-CCM encrypted      CCM tag
```

Byte-level table, sealing and receiver checks: [SECURITY.md §7-8](#). `tools/gps_lora.py decode MASTER_FILE <hex>` decrypts packets captured by other receivers.

Security

In short ([full specification](#)):

- **Protected:** positions over the air and on the tracker's SD card (AES-128-CCM, 8-byte tag); forged, altered, replayed and stale packets are rejected; a captured tracker exposes only its own data (per-tracker keys derived from your master key with HKDF-SHA256).

- **Not protected:** that a tracker is transmitting and roughly where; keys in flash against someone who dumps a board (until eFuse hardening); decrypted data on the receiver; the plain-HTTP web page.
- **Fail closed:** no key, no transmission. Self-test failure at boot, no encryption. Counter cannot be saved, record dropped rather than written in clear.

Provisioning

Needs `pip install cryptography`. Once, on your PC (keep the file offline and backed up; losing it means losing access to every log):

```
tools/gps_lora.py new-master ~/gps-lora.master
tools/gps_lora.py provision ~/gps-lora.master ab12      # each tracker, node ID from `status`
tools/gps_lora.py provision ~/gps-lora.master receiver  # the receiver
```

Paste each printed `key set ...` line into that board's console; after the reboot `status` must show the same fingerprint. Then clear the terminal scrollbar. Details and procedures for lost keys and boards: [SECURITY.md §11-12](#).

Console

Connect with `arduino-cli monitor -p /dev/ttyACM0` (or any terminal) and type `help`:

Command	Effect
<code>status</code>	GNSS, SD, LoRa, WiFi state
<code>wifi ssid <name></code>	Network to join (rest of line, spaces allowed)
<code>wifi pass <password></code>	Network password (never echoed)
<code>web pass <password></code>	Web page password, 8+ characters, user <code>admin</code>
<code>wifi on / wifi off</code>	Service mode
<code>key set <32 hex></code>	Install the key from <code>tools/gps_lora.py provision</code> , reboots
<code>key clear</code>	Remove the key, reboots
<code>gnss reset</code>	UBX-CFG-RST hot start
<code>eject / resume</code>	SD card
<code>reboot</code>	Closes the log file first

Settings are kept in NVS in plain text (no flash encryption).

WiFi service mode

Off by default. Turn it on with a **short press of the eject button** or `wifi on`. It needs a network and a web password set first. The OLED's last line shows the IP address; the board is also at `http://gps-lora-<node>.local/`.

Page	
<code>/</code>	Status, log list with download and delete, firmware upload
<code>/status</code>	Status as JSON
<code>/download?f=LOG0001.CSV</code>	Download a log, including the one being written

Page	
/update	POST a firmware image (tracker.ino.bin or receiver.ino.bin)

All pages need HTTP basic auth (admin and the web password). This is plain HTTP, so use it on a network you trust. WiFi switches itself off after 10 minutes without a request, or after 30 s if it cannot connect. A long download does not stop logging: the SD lock is taken per 2 KB chunk.

While WiFi is on, the system clock is also set over NTP (GNSS sets it otherwise), so SD files get correct timestamps even without a fix.

OTA update and rollback

Upload tracker.ino.bin or receiver.ino.bin on the web page; the page title shows which firmware the board is running. Uploading the other one turns a tracker into a receiver or back. The board reboots into the new image, which must then run for **60 s** before it is marked good. If it crashes, hangs (the loop watchdog fires after 5 s) or loses power within those 60 s, the bootloader starts the previous image again. So keep the board powered for a minute after an update.

Receiver

Flash receiver/ onto a second board and give it the master key. It still runs its own GNSS, so with both ends fixed the OLED shows distance, compass direction and RSSI to the tracker. One receiver hears any number of trackers; it derives each one's key from the master. Every authenticated packet is appended to RXnnnn.CSV (decrypted):

```
rx_utc,node,counter,unix_time,lat,lon,alt_m,speed_kmh,sats,hdop,flags,rssi_dbm,snr_db
```

and printed on USB as one JSON line (lines starting with {):

```
{"rx_utc":"2026-10-07T09:12:45Z","node":"ab12","counter":96432110,...,"rssi_dbm":-97.0,"snr_db":8.25}
```

Rejected frames are printed as rx: dropped ... (authentication failed | replay | stale | ...) and counted as rej on the OLED.

Status LED

Pattern	Meaning
5 fast blinks, repeating	No card, or card error (remount retried every 5 s)
Short blip every 2 s	Card OK, waiting for a fix (tracker) or a packet (receiver)
Blip on each record	Logging
Solid on	Card ejected, safe to remove

Eject button: short press toggles WiFi. Hold 1 s to close the file and unmount the card; press again to remount and start a new file.

Not done yet

- Receiver firmware for boards other than gps-lora (only the pin map differs)
- Forwarding received packets to a server over WiFi
- Low-power handling of the GNSS

- eFuse hardening: flash encryption, NVS encryption, secure boot ([SECURITY.md §16](#))

Bring-up order

1. USB enumerates, boot banner on the serial monitor, `security: self-test passed` (this is the first run of the crypto on real hardware)
2. GNSS: `gnss: messages stop`, OLED shows time, then PPS
3. SD: `sd: mounted`, file appears after the first fix
4. Provision keys; `status` shows the expected fingerprints
5. LoRa: `lora: 866.0 MHz ...`; check with a second radio or SDR
6. OLED (panel fitted at final assembly)
7. WiFi: set credentials, `wifi on`, open the page, download a log
8. OTA: upload the same build, check `ota: new firmware confirmed` after 60 s
9. Second board with the receiver firmware: packets arrive, distance looks right; `tools/gps_lora.py d`
`encrypt-log` reads the tracker's card

License

Apache License 2.0, see [LICENSE](#). The hardware in `hardware/` is CERN-OHL-P-2.0.